



## **INSTITUTO CATÓLICO KAROL WOJTYLA**

### **Actividad desarrollada:**

Preparación para la Integración App - Base de Datos

### **Grado:**

Tercer Año de Bachillerato Técnico en Desarrollo de Software

### **MÓDULO 3.3:** Administración de Bases de Datos

### **Integrantes:**

José Rolando Velasco Peña

Luis Mario Meléndez Escobar

Ulises de Jesús Mercado Alberto

### **Docente:**

Lic. Criseyda Guadalupe Araujo Mengar

Cabañas Oeste, El Salvador

Lunes 28 de septiembre de 2026



## ÍNDICE

|                                                       |    |
|-------------------------------------------------------|----|
| ÍNDICE .....                                          | 2  |
| TABLA DE FIGURAS .....                                | 3  |
| 1. DESCRIPCIÓN DEL PROYECTO.....                      | 4  |
| 2. FUNCIONES SELECCIONADAS DE LA APLICACIÓN.....      | 5  |
| 2.1. Identificación y autenticación del usuario.....  | 5  |
| 2.1.1. Información necesaria: .....                   | 6  |
| 2.1.2. Tabla solicitada y operación SQL.....          | 6  |
| 2.2. Registro de alertas .....                        | 7  |
| 2.2.1. Información necesaria: .....                   | 7  |
| 2.2.2. Tabla solicitada y operación SQL.....          | 7  |
| 2.3. Consulta de alertas .....                        | 7  |
| 2.3.1. ¿Qué información necesita?.....                | 8  |
| 2.3.2. Tabla solicitada y operación SQL.....          | 8  |
| 2.4. Consulta y actualización de estados .....        | 8  |
| 2.4.1. ¿Qué información necesita?.....                | 9  |
| 2.4.2. Tabla solicitada y operación SQL.....          | 9  |
| 3. TABLA DE CORRESPONDENCIA APP - BASE DE DATOS ..... | 9  |
| 4. OPERACIONES SQL RELACIONADAS .....                 | 11 |
| 4.1. Operaciones SQL.....                             | 11 |
| 5. INFORMACIÓN QUE DEBE PROTEGERSE.....               | 17 |
| 6. DIAGRAMA DE INTEGRACIÓN.....                       | 17 |
| 7. FLUJO DE UNA FUNCIÓN PRINCIPAL.....                | 20 |
| 8. RESPONSABILIDAD DE CADA INTEGRANTE .....           | 23 |
| 9. EVIDENCIAS DEL TRABAJO .....                       | 23 |
| 9.1. PostgreSQL.....                                  | 23 |
| 9.2. GitHub.....                                      | 27 |
| 10. CONCLUSIÓN .....                                  | 30 |



## TABLA DE FIGURAS

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| Figura 1 Diagrama de integración de SafeSchool Alert y flujo de datos .....               | 19 |
| Figura 2 Diagrama de Flujo de datos para el registro de alertas de SafeSchool Alert ..... | 22 |
| Figura 3 Captura de evidencia de tabla alertas en PostgreSQL .....                        | 23 |
| Figura 4 Captura de evidencia de tabla instituciones en PostgreSQL .....                  | 24 |
| Figura 5 Captura de evidencia de tabla ubicaciones en PostgreSQL .....                    | 25 |
| Figura 6 Captura de evidencia de tabla tipos_emergencia en PostgreSQL .....               | 25 |
| Figura 7 Captura de evidencia de tabla usuarios en PostgreSQL .....                       | 26 |
| Figura 8 Captura de evidencia de tabla historial en PostgreSQL .....                      | 27 |
| Figura 9 Captura del repositorio del proyecto en GitHub .....                             | 28 |
| Figura 10 Captura de las ramas de cada uno de los integrantes del equipo .....            | 29 |
| Figura 11 Captura de lista de commits de los integrantes .....                            | 29 |



## **1. DESCRIPCIÓN DEL PROYECTO**

### **1.1. ¿Qué es SafeSchool Alert?**

La aplicación móvil SafeSchool Alert es un sistema inteligente de alertas y emergencias escolares, el cual es capaz de detectar y darle un seguimiento y atención a alertas de tipo médica, seguridad y de incendio gracias al trabajo en conjunto con el microcontrolador Arduino Uno, que es el encargado de activar y enviar las alertas de emergencia a la aplicación móvil por medio de comunicación serial a través de Bluetooth. Al momento que la aplicación detecta una emergencia en la institución educativa, informa al personal restante de la institución, tanto a docentes, administrativo y estudiantil, sobre la emergencia que se ha detectado o informado, por medio de la aplicación móvil y mensajes SMS para una atención rápida y seguimiento a dicha emergencia.

El sistema tiene como propósito darle solución al problema de la lentitud y la ineficiencia de la comunicación ante emergencias que ponen en riesgo el bienestar y la seguridad de la población de la institución, con una aplicación móvil que detecta, informa, gestiona y promociona información de cada una de las emergencias detectadas.

### **1.2. ¿Qué problema resuelve?**

SafeSchool Alert busca solucionar el problema de la comunicación lenta e ineficiente que se da en las instituciones del sector educativo al momento de una emergencia, dentro de la institución, ya que muchas veces la información sobre una emergencia puede tardar en llegar al personal de atención. Por medio del sistema se busca informar las emergencias detectadas de carácter médicas, seguridad y incendio pueda ser atendidas y gestionadas de una manera más rápida, permitiendo que el personal pueda conocer el tipo de emergencia y el lugar donde está ocurriendo para darle una solución y saber guardar la seguridad y bienestar de toda la población.

### **1.3. ¿Qué hace la aplicación?**

La aplicación móvil permite recibir las alertas de emergencia que son enviadas por el Arduino Uno, mostrando información como el tipo de emergencia, la ubicación donde está ocurriendo y el estado en el que se encuentra. También permite llevar un registro de historial de



las emergencias ocurridas por medio de un historial, para poder consultar la información de las alertas que han sido registradas anteriormente y dar un mejor seguimiento a cada emergencia.

#### **1.4. ¿Qué información necesita almacenar y consultar?**

La aplicación almacena y consulta información relacionada con una de las emergencias que se detectan dentro de la institución educativa, el tipo de emergencia que se ha reportado o detectado, la ubicación en que se originó la emergencia en la institución, el estado de la emergencia, la fecha y hora en que ocurrió y el comando recibido por el Aduino. También necesita almacenar la información de los usuarios que utilizan la aplicación, para poder identificar a cada usuarios y personal actualizado para acceder a las diferentes funciones del sistema según su tipo rol.

#### **1.5. ¿Qué utiliza PostgreSQL como base de datos?**

SafeSchool Alert utiliza PostgreSQL como sistema de base de datos para almacenar y consultar la información que necesita la aplicación para su funcionamiento. En esta base de datos se almacenan los datos relacionados con las alertas, los tipos de emergencia, las ubicaciones y los usuarios, permitiendo tener la información organizada y disponible para realizar las diferentes consultas que necesita el sistema.

## **2. FUNCIONES SELECCIONADAS DE LA APLICACIÓN**

Con el propósito de comprender e identificar las necesidades reales y operaciones SQL principales que el sistema necesita ejecutar para obtener y gestionar la información de cada una de las emergencias, en esta sección se presentan 4 funciones de consulta de datos a la BD que el sistema necesitará conocer, para gestionar y darle un seguimiento correcto a cada una de las emergencias que se detectan en la institución educativa.

### **2.1. Identificación y autenticación del usuario**

Esta función es una de las principales funciones de consulta de datos que el sistema realizará a la BD, para identificar a cada uno de los usuarios que tienen permisos de acceso al sistema para atender y gestionar cada una de las emergencias. Su función principal es autenticar



que el usuario que desea ingresar al sistema es quien dice ser. Además, el sistema necesita saber y recordar la información de cada usuario, para consultar sus datos personales almacenados como: nombre, correo, institución, rol, etc., y utilizarlo para cada una de las emergencias, como el usuario que atendió la emergencia o el que la reportó.

#### **2.1.1. Información necesaria:**

La información principal que el sistema necesita saber y consultar para esta función del sistema, son los relacionados con los datos personales y de autenticación de cada usuario los cuales son los siguientes: correo personal del usuario, el identificador único del usuario, nombre completo del usuario, número telefónico, fecha de registro, institución educativa perteneciente, estado etc.

#### **2.1.2. Tabla solicitada y operación SQL**

La selección en la cual se obtendrán estos datos de cada usuario registrado en el sistema proviene de la tabla “usuarios”, la cual almacena los datos personales y de contacto de cada uno de los usuarios. No obstante, no basta con conocer al usuario, sino que también a qué institución educativa pertenece.

Las operaciones SQL necesarias para consultar esta información a la base de datos son las siguientes: SELECT e INNER JOIN. Son las operaciones necesarias principalmente, para realizar una consulta de datos a la BD y obtener, además, los campos relacionados con el usuario de la tabla “instituciones”, para conocer a qué institución pertenece el usuario. Como también para consultar los datos de autenticación del usuario como lo son el correo y la contraseña para poder ingresar al sistema.



## **2.2. Registro de alertas**

Esta función se encarga de registrar cada una de las emergencias que el sistema detecta en la institución educativa. El registro de las alertas se realiza luego de que la aplicación recibe el comando de la emergencia activada desde el Arduino por medio de Bluetooth. Luego, el sistema identifica el tipo de emergencia y obtiene la información necesaria para registrar la alerta en la BD, permitiendo que esta pueda ser consultada posteriormente para darle seguimiento a la emergencia.

### **2.2.1. Información necesaria:**

Para poder registrar correctamente cada alerta, el sistema necesita obtener información como el tipo de emergencia, el estado en el que se encuentra, identificar la ubicación donde fue detectada, guardar la fecha y hora en que ocurrió la emergencia y el comando recibido desde el Arduino.

### **2.2.2. Tabla solicitada y operación SQL**

La tabla en la cual se almacenará esta información es la tabla “alertas”, ya que esta es la encargada de guardar los datos relacionados con cada una de las emergencias detectadas por el sistema. En esta tabla se registrará el tipo de emergencia, el estado, la ubicación, la fecha y hora y el comando recibido.

Las operaciones SQL necesarias para realizar esta función son principalmente INSERT y SELECT. La operación INSERT será utilizada para registrar los datos de la nueva alerta en la BD luego de que el sistema reciba y procese el comando enviado desde el Arduino.

## **2.3. Consulta de alertas**

Esta función se encarga de consultar las alertas que han sido registradas en el sistema, esta información es de vital importancia ya al realizar esta consulta de datos, permite que el sistema pueda mostrar las alertas registradas y conocer información como el tipo de emergencia, el estado, la ubicación y la fecha y hora en que ocurrió, para poder darle un seguimiento correcto a cada una de las emergencias.



### **2.3.1. ¿Qué información necesita?**

Para realizar esta consulta, el sistema necesita conocer la información almacenada de cada una de las alertas, como el tipo de emergencia, el estado en el que se encuentra, la ubicación donde ocurrió y la fecha y hora en que fue registrada. También el sistema necesita identificar al usuario que atendió y dio un seguimiento a cada una de las alertas.

### **2.3.2. Tabla solicitada y operación SQL**

La tabla que se consultara para obtener los datos almacenados de cada una de las emergencias repostados, es la tabla “alertas”, ya que esta contiene la información de cada una de las emergencias registradas por el sistema. De igual manera se deberá consultar otras tablas relacionadas, como “tipos\_emergencia” para conocer el tipo de la emergencia, “ubicaciones” para conocer en qué lugar de la institución se generó la emergencia y “usuarios”, para obtener la información como el nombre del usuario que atendió la emergencia.

Las operaciones SQL necesarias para realizar esta consulta son principalmente SELECT e INNER JOIN, ya que SELECT permitirá obtener los datos de las alertas registradas en la BD, mientras que INNER JOIN permitirá relacionar la información de la tabla “alertas” con las demás tablas relacionadas para obtener los datos necesarios de cada emergencia.

### **2.4. Consulta y actualización de estados**

La función de consulta y actualización de estados se refiere a la consulta y actualización del estado de una emergencia que se ha detectado en la institución educativa. Esta función es de vital importancia para el sistema, ya que por medio de esta el sistema puede darle un seguimiento a cada emergencia según el estado en que se encuentre. Además de consultar los estados de emergencia, el sistema necesitará actualizar los estados una vez que un usuario atienda y dé seguimiento a una emergencia, permitiendo conocer si esta se encuentra activa, en proceso, atendida o si fue una falsa alarma.





### 2.4.1. ¿Qué información necesita?

Para realizar esta función, el sistema necesitara consultar la información del estado en el que se encuentra la emergencia, para darle un seguimiento correcto a cada una de las emergencias. También necesita conocer el usuario que está atendiendo la emergencia, para poder actualizar la información.

### 2.4.2. Tabla solicitada y operación SQL

La tabla en la cual se encuentra almacenada esta información es la tabla “alertas”, ya que esta contiene el estado actual de cada una de las emergencias registradas en el sistema. Como también se necesaria consultar la tabla “usuarios” para identificar el usuario que está atendiendo o dando seguimiento cada emergencia.

Las operaciones SQL necesarias para realizar esta función son principalmente SELECT y UPDATE. La operación SELECT permitirá consultar el estado actual de cada alerta, y UPDATE permitirá actualizar el estado de la emergencia una vez que un usuario la atienda, o actualice su estado.

## 3. TABLA DE CORRESPONDENCIA APP - BASE DE DATOS

| Función de la aplicación                         | Tabla PostgreSQL         | Operación SQL         | Información utilizada                                                                                                                                  |
|--------------------------------------------------|--------------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Autenticación de usuarios</b>                 | usuarios                 | SELECT                | Los datos del usuario que necesita el sistema para realizar la autenticación son: correo, contraseña, rol e institución a la que pertenece el usuario. |
| <b>Registro de usuarios nuevos en el sistema</b> | usuarios                 | INSERT                | Los datos que necesitará guardar el sistema en el registro son: nombre, correo, contraseña, teléfono, rol e institución educativa.                     |
| <b>Consulta de datos de usuarios</b>             | usuarios e instituciones | SELECT con INNER JOIN | Los datos que el sistema solicitará son: nombre, correo, teléfono, institución, rol y fecha de creación.                                               |



|                                            |                                                              |                       |                                                                                                                                                                                                       |
|--------------------------------------------|--------------------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cantidad de docentes registrados</b>    | usuarios                                                     | SELECT y COUNT()      | Registros de la tabla usuarios donde el rol del usuario sea igual a “docente”, para realizar el conteo de docentes registrados.                                                                       |
| <b>Cantidad de usuarios registrados</b>    | usuarios                                                     | SELECT y COUNT()      | Registros de la tabla usuarios para realizar el conteo exacto de usuarios registrados en el sistema.                                                                                                  |
| <b>Cantidad de emergencias</b>             | alertas                                                      | SELECT y COUNT()      | Registros de la tabla alertas para realizar el conteo exacto de las emergencias registradas en el sistema.                                                                                            |
| <b>Cantidad de ubicaciones</b>             | ubicaciones                                                  | SELECT y COUNT()      | Registros de la tabla ubicaciones para realizar el conteo exacto de la cantidad de ubicaciones registradas en la institución educativa.                                                               |
| <b>Cantidad de registros de historial</b>  | historial                                                    | SELECT y COUNT()      | Registros de la tabla historial para realizar el conteo de los eventos y emergencias que han sido registrados en el sistema.                                                                          |
| <b>Consulta de registros del historial</b> | historial, usuarios, alertas, tipos_emergencia y ubicaciones | SELECT con INNER JOIN | Los datos necesarios para crear un registro del historial de eventos de las emergencias son: nombre de la emergencia, tipo de emergencia, ubicación, descripción, usuario y fecha y hora de registro. |
| <b>Registro de alertas</b>                 | alertas                                                      | INSERT                | Los datos principales para el registro de alertas son: usuario, tipo de emergencia, ubicación, comando recibido, descripción, estado y fecha y hora de la emergencia.                                 |
| <b>Consulta de alertas</b>                 | alertas, tipos_emergencia, ubicaciones y usuarios            | SELECT con INNER JOIN | Los principales campos que serán utilizados por el sistema son: id de alerta, tipo de emergencia, ubicación, estado, usuario, fecha y hora detectada, comando recibido y descripción.                 |



|                                        |                  |        |                                                                                                                                                                                    |
|----------------------------------------|------------------|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Consultar tipos de emergencia</b>   | tipos_emergencia | SELECT | Los campos solicitados son: id del tipo, tipo de emergencia y fecha de creación.                                                                                                   |
| <b>Registrar un tipo de emergencia</b> | tipos_emergencia | INSERT | Los datos que necesitará guardar el sistema son: tipo de emergencia y fecha de creación.                                                                                           |
| <b>Consultar ubicaciones</b>           | ubicaciones      | SELECT | Los datos utilizados por el sistema son: id de ubicación, nombre del edificio, institución y fecha de creación.                                                                    |
| <b>Registrar una ubicación</b>         | ubicaciones      | INSERT | Los datos necesarios para registrar una nueva ubicación son: institución, nombre de la ubicación y fecha de creación.                                                              |
| <b>Consultar institución educativa</b> | instituciones    | SELECT | Los datos utilizados por el sistema son: id de institución, nombre de la institución y fecha de creación.                                                                          |
| <b>Registrar institución educativa</b> | instituciones    | INSERT | Los datos que necesitará guardar el sistema son: nombre de la institución y fecha de creación.                                                                                     |
| <b>Actualizar estado de alerta</b>     | alertas          | UPDATE | El campo estado de la alerta activa en el sistema, el cual puede ser actualizado cuando la emergencia se encuentre en proceso, haya sido atendida o sea marcada como falsa alarma. |

## 4. OPERACIONES SQL RELACIONADAS

### 4.1. Operaciones SQL

#### 4.1.1. Función: Autenticación de usuarios

##### Operación:

- **SELECT:** Se utiliza para consultar los datos ingresados por el usuario y verificar si existe un registro que coincida con las credenciales almacenadas, permitiendo determinar si puede ingresar al sistema.



**Ejemplo:** Cuando un usuario intenta iniciar sesión, el sistema consulta la base de datos para comprobar si sus credenciales son correctas.

#### 4.1.2. Función: Registro de usuarios nuevos en el sistema

##### Operación:

- **INSERT:** Se utiliza para agregar un nuevo registro a la tabla de usuarios, almacenando los datos necesarios para que posteriormente puedan ser consultados y validados.

**Ejemplo:** Cuando se crea una nueva cuenta, el sistema almacena los datos del usuario para que posteriormente puedan ser consultados o validados.

#### 4.1.3. Función: Consulta de datos de usuarios.

##### Operaciones:

- **SELECT:** Se utiliza para recuperar y mostrar los datos almacenados de los usuarios.
- **INNER JOIN:** Se utiliza para relacionar la información de los usuarios con la tabla de instituciones mediante los campos relacionados, permitiendo mostrar información de ambas tablas en una sola consulta.

**Ejemplo:** Cuando el administrador consulta un usuario, el sistema obtiene sus datos y los relaciona con la institución correspondiente para mostrar la información de manera completa.

#### 4.1.4. Función: Cantidad de docentes registrados.

##### Operaciones:

- **SELECT:** Se utiliza para realizar la consulta que obtiene la cantidad de docentes registrados.
- **COUNT ():** Se utiliza para contar los registros de usuarios que tienen asignado el rol de docente y mostrar el resultado.

**Ejemplo:** Cuando el administrador necesita conocer cuántos docentes están registrados, el sistema consulta los usuarios y cuenta únicamente aquellos que tienen el rol de docente.



#### 4.1.5. Función: Cantidad de usuarios registrados.

##### Operaciones:

- **SELECT:** Se utiliza para consultar los registros almacenados en la tabla de usuarios.
- **COUNT ():** Se utiliza para contar todos los usuarios registrados y obtener el total que será mostrado en el sistema.

**Ejemplo:** Cuando se necesita conocer el total de usuarios del sistema, se consultan los registros y se cuentan para mostrar la cantidad total.

#### 4.1.6. Función: Cantidad de emergencias

##### Operaciones:

- **SELECT:** Se utiliza para consultar los registros almacenados en la tabla de alertas.
- **COUNT ():** Se utiliza para contar la cantidad total de emergencias registradas en la tabla de alertas.

**Ejemplo:** Cuando el administrador necesita conocer cuántas emergencias han sido registradas, el sistema consulta las alertas y cuenta todos los registros existentes para poder mostrarlos en la aplicación.

#### 4.1.7. Función: Cantidad de ubicaciones.

##### Operaciones:

- **SELECT:** Se utiliza para consultar los registros almacenados en la tabla ubicaciones.
- **COUNT ():** Se utiliza para contar la cantidad total de ubicaciones registradas en la base de datos.

**Ejemplo:** Cuando se necesita conocer cuántas ubicaciones están registradas en el sistema, se consultan y cuentan los registros de la tabla de ubicaciones para utilizar estos datos en la aplicación.



#### 4.1.8. Función: Cantidad de registros de historial.

##### Operaciones:

- **SELECT:** Se utiliza para consultar los registros almacenados en la tabla historial.
- **COUNT ():** Se utiliza para contar la cantidad total de registros existentes en el historial.

**Ejemplo:** Cuando el administrador necesita conocer cuántos eventos se encuentran almacenados en el historial, el sistema consulta y cuenta sus registros para mostrarlos en el apartado historial de la aplicación.

#### 4.1.9. Función: Consulta de registros del historial.

##### Operaciones:

- **SELECT:** Se utiliza para recuperar y mostrar los registros almacenados en el historial.
- **INNER JOIN:** Se utiliza para relacionar las tablas: historial, usuarios, alertas, tipos\_emergencia y ubicaciones, permitiendo obtener información relacionada en una sola consulta.

**Ejemplo:** Cuando el administrador revisa el historial de emergencias, el sistema relaciona las diferentes tablas para mostrar información completa sobre el usuario, la alerta, el tipo de emergencia y la ubicación.

#### 4.1.10. Función: Registro de alertas.

##### Operación:

- **INSERT:** Se utiliza para agregar una nueva alerta a la base de datos, almacenando los datos correspondientes a la emergencia registrada.

**Ejemplo:** Cuando un usuario registra una emergencia, el sistema almacena la nueva alerta en la base de datos.

#### 4.1.11. Función: Consulta de alertas.

##### Operaciones:

- **SELECT:** Se utiliza para recuperar y mostrar las alertas registradas en la base de datos.



- **INNER JOIN:** Se utiliza para relacionar las tablas alertas, tipos\_emergencia, ubicaciones y usuarios, permitiendo obtener información adicional de cada alerta.

**Ejemplo:** Cuando el administrador consulta una alerta, el sistema obtiene sus datos y los relaciona con el tipo de emergencia, la ubicación y el usuario que la registró.

#### 4.1.12. Función: Consultar tipos de emergencias.

##### Operación:

- **SELECT:** Se utiliza para consultar los tipos de emergencias almacenados en la base de datos, mostrando los campos necesarios como el identificador, tipo de emergencia y fecha.

**Ejemplo:** Cuando el sistema necesita mostrar los tipos de emergencia disponibles, consulta los registros correspondientes y presenta su identificador, tipo y fecha.

#### 4.1.13. Función: Registrar un tipo de emergencia.

##### Operación:

- **INSERT:** Se utiliza para agregar un nuevo tipo de emergencia a la base de datos, permitiendo que pueda ser utilizado posteriormente en el registro de alertas.
- **Ejemplo:** Cuando el administrador necesita agregar un nuevo tipo de emergencia, el sistema almacena sus datos para que pueda utilizarse posteriormente en la aplicación.

#### 4.1.14. Función: Consultar ubicaciones

##### Operación:

- **SELECT:** Se utiliza para recuperar las ubicaciones registradas en la base de datos, mostrando los campos necesarios como el identificador, nombre, institución relacionada y fecha.

**Ejemplo:** Cuando se necesita consultar las ubicaciones disponibles, el sistema recupera sus datos para mostrarlos en pantalla.

#### 4.1.15. Función: Registrar ubicación.



#### Operación:

- **INSERT:** Se utiliza para agregar una nueva ubicación a la base de datos, almacenando la información necesaria para identificarla y relacionarla con la institución correspondiente.

**Ejemplo:** Cuando se incorpora una nueva ubicación de la institución, el sistema almacena sus datos para poder utilizarla posteriormente en el registro de una alerta.

#### 4.1.16. Función: Consultar institución educativa.

##### Operación:

- **SELECT:** Se utiliza para consultar y recuperar la información almacenada en la tabla instituciones, permitiendo mostrar los datos de las instituciones educativas registradas.

**Ejemplo:** Cuando el sistema necesita mostrar las instituciones educativas registradas, consulta la información almacenada en la base de datos.

#### 4.1.17. Función: Registrar institución educativa.

##### Operación:

- **INSERT:** Se utiliza para agregar una nueva institución educativa a la base de datos, almacenando sus datos para que puedan ser utilizados posteriormente por el sistema.

**Ejemplo:** Cuando se registra una nueva institución, el sistema almacena sus datos para que puedan utilizarse posteriormente.

#### 4.1.18. Función: Actualizar estado de alerta.

##### Operación:

- **UPDATE:** Se utiliza para modificar el estado de una alerta que ya se encuentra registrada en la base de datos, permitiendo actualizar el valor del campo id\_estado sin crear un nuevo registro.

**Ejemplo:** Cuando cambia el estado de una emergencia, el sistema actualiza el estado de la alerta existente para reflejar su nueva situación.





## 5. INFORMACIÓN QUE DEBE PROTEGERSE

### 5.1. ¿Qué información debemos proteger?

- Contraseñas de usuarios
- Información de usuarios
- Credenciales de administrador

### 5.2. ¿Por qué debemos Protegerla?

Es necesario proteger la información antes mencionada para asegurar la privacidad de los usuarios, así mismo es necesario ocultar las credenciales de administradores para que una persona exterior a la institución no interfiera con las alertas del sistema o modifique algún estado y de esta forma generar confianza en los usuarios.

### 5.3. ¿Qué medidas de seguridad aplicaríamos?

Principalmente se ocultarían las credenciales tanto de usuarios normales como de los administradores dentro de la base de datos y controlar quienes tienen acceso a ella para evitar que ocurran filtraciones de información.

## 6. DIAGRAMA DE INTEGRACIÓN

El diagrama de integración permite visualizar de forma gráfica y rápida el flujo principal de integración entre el sistema (Arduino + aplicación móvil) y la base de datos. Este representa cómo se comunican los diferentes módulos para intercambiar, procesar y entregar datos. La comunicación entre los componentes se realiza mediante el protocolo HTTPS por medio de una API REST, encargada de establecer la comunicación e intercambio de datos con la base de datos y el sistema.

- El flujo inicia con la activación de una emergencia mediante los pulsadores o de forma automática mediante el sensor MQ-2, activando los componentes electrónicos que alertan sobre la emergencia detectada.



- Posteriormente, la emergencia reportada en la institución es comunicada a la aplicación móvil, donde se determina el tipo de emergencia, la ubicación, fecha, hora y el estado en que se encuentra.
- Seguidamente, los datos y consultas provenientes de la aplicación son enviados a la API REST, la cual realiza las validaciones necesarias, prepara la consulta y ejecuta la operación SQL correspondiente en la base de datos.
- Finalmente, la API realiza las peticiones a la base de datos y recibe los resultados. Estos son preparados en formato JSON para ser enviados a la aplicación móvil y mostrados en pantalla.

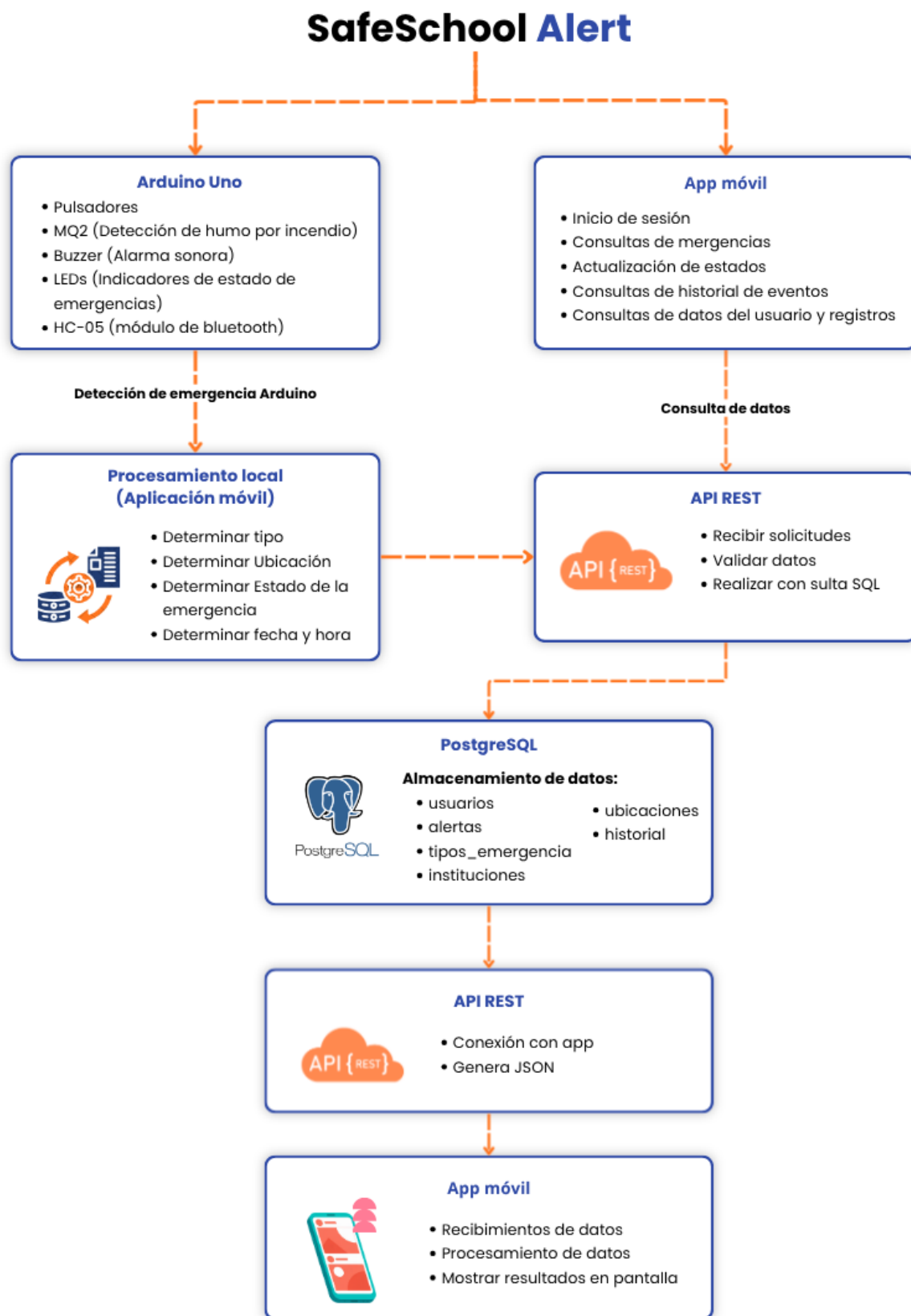


Figura 1 Diagrama de integración de SafeSchool Alert y flujo de datos



## 7. FLUJO DE UNA FUNCIÓN PRINCIPAL

### 7.1. Registrar una alerta

Para representar cómo viajará la información desde el Arduino hacia la aplicación y la API, se decidió representar el flujo de información necesario para el registro de una emergencia, hasta que esta sea comunicada al resto de la población educativa y mostrada en pantalla como una emergencia activa dentro de la institución educativa. Finalmente, la emergencia podrá ser atendida tomando las acciones correspondientes según su estado y tipo.

Para ello, se diseñó el flujo de datos que recorre el sistema para el registro de una emergencia, representado mediante un diagrama de flujo. En este se muestran las validaciones y procesos necesarios que se ejecutan en el sistema para cumplir con la función de comunicar rápidamente al personal docente, estudiantil y administrativo sobre la emergencia que se encuentra activa en la institución educativa.

### 7.2. Explicación de cada etapa:

- **Etapas de activación:** El flujo arranca con la activación de una emergencia, ya sea manualmente por medio de los pulsadores o mediante los sensores de gas. Una vez activada la emergencia, el Arduino identifica el tipo de emergencia y la ubicación en la que se ha generado. Posteriormente, activa la alarma sonora, enciende los LED indicadores y muestra mensajes en la pantalla LCD.
- **Etapas de reporte de la emergencia a la app:** Una vez que la emergencia es activada, el Arduino prepara el comando de emergencia siguiendo el protocolo de comunicación establecido. Seguidamente, se envía a la aplicación, donde se identifica el tipo, la ubicación y el estado de la emergencia para preparar la información que será enviada a la API.
- **Etapas de solicitud a la API:** La aplicación prepara y envía una solicitud a la API para registrar la emergencia. En este proceso se envían los datos necesarios para realizar el



registro en la base de datos. La API recibe la solicitud y verifica que haya sido recibida correctamente. Si ocurre algún error, la aplicación muestra un mensaje y permite volver a realizar el proceso.

- **Etapas de validación de datos:** Una vez, recibida la solicitud, la API valida los datos enviados, comprobando que cumplan con el formato requerido y las condiciones de seguridad establecidas. Si los datos no son válidos la API rechaza la solicitud, y devuelve un mensaje de error a la aplicación. Si los datos son correctos, el proceso continúa con el registro de la emergencia.
- **Etapas de registro en la base de datos:** Después se valida la información recibida, la API ejecuta la operación INSERT en la tabla “alertas” de PostgreSQL. La base de datos procesa la operación, y registra la información de la emergencia, dejando actualizada la tabla con los datos correspondientes.
- **Etapas de respuesta de la API:** Una vez que PostgreSQL termina la operación, está regresa el resultado a la API. Esta recibe la respuesta de la base de datos y genera una respuesta en formato JSON, la cual contiene el resultado del registro de la emergencia.
- **Etapas de procesamiento en la aplicación:** La aplicación recibe los datos enviados por la API y guarda los datos en memoria para utilizarlos durante la ejecución del sistema. Después, procesa la información y realiza las acciones correspondientes según el tipo, ubicación y estado de la emergencia registrada.
- **Etapas de notificación y visualización:** Posteriormente la aplicación comunica a los usuarios de la emergencia activa y muestra en pantalla sus detalles, con las opciones disponibles para realizar una gestión rápida de la emergencia.
- **Etapas de finalización:** Una vez que este proceso termina, el sistema queda pendiente de la activación de una nueva emergencia para su registro y comunicación.

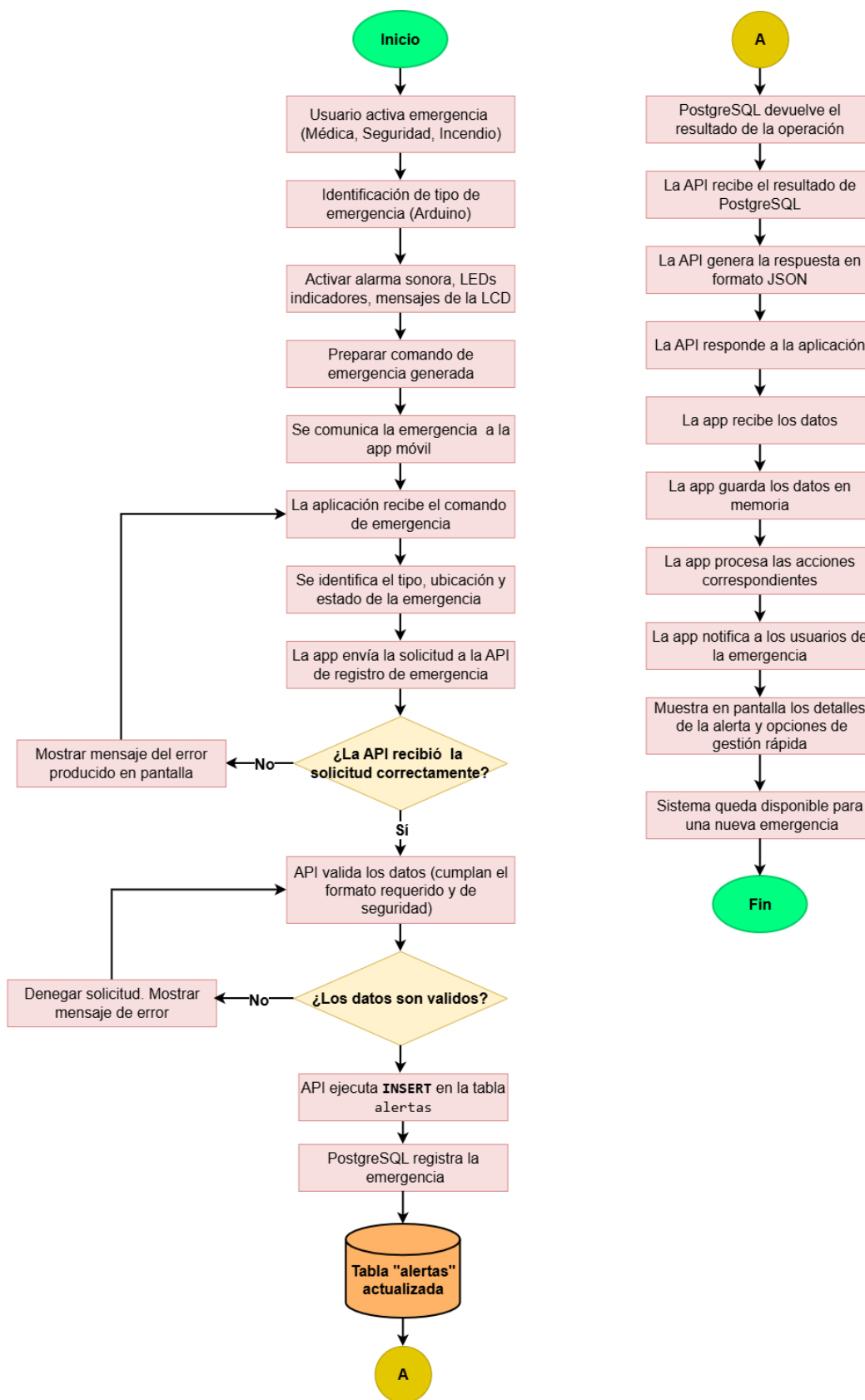


Figura 2 Diagrama de Flujo de datos para el registro de alertas de SafeSchool Alert

## 8. RESPONSABILIDAD DE CADA INTEGRANTE

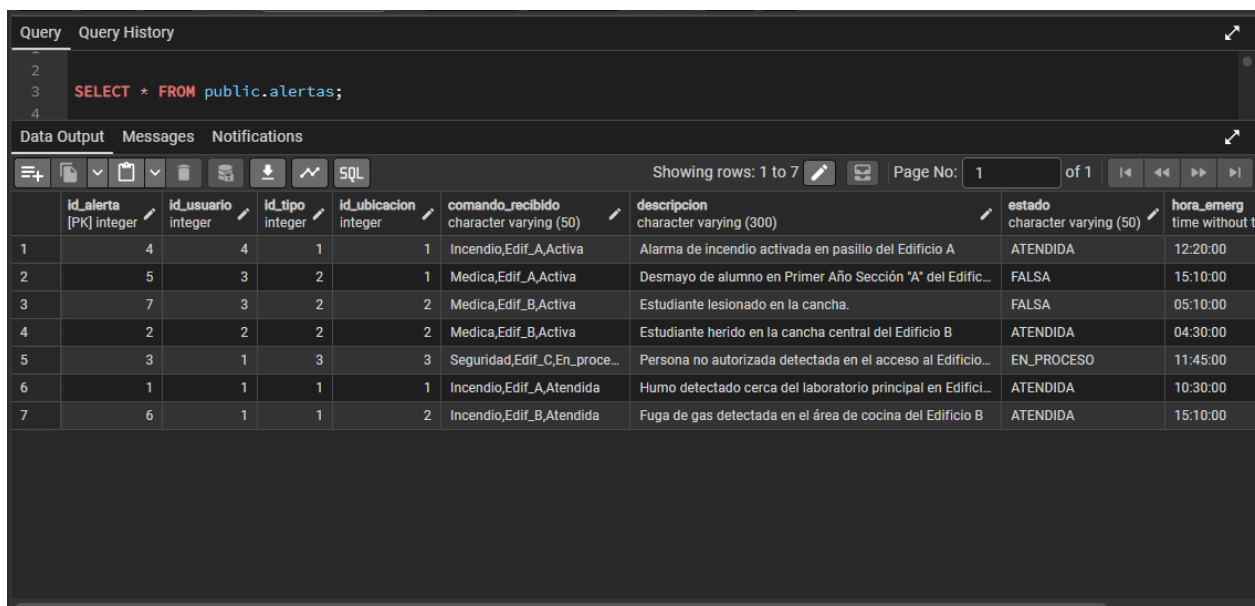
## 9. EVIDENCIAS DEL TRABAJO

Con el propósito de evidenciar la existencia de las tablas y los datos almacenados en cada una de ellas, que han sido mencionadas en las funciones de la tabla de correspondencia y funciones SQL, en este apartado se presentan las evidencias de cada una de las tablas mencionadas: alertas, instituciones, ubicaciones, tipos\_emergencia, usuarios e historial, por medio de capturas de pantalla que muestran las tablas y los datos almacenados en cada una de ellas.

### 9.1. PostgreSQL

#### 9.1.1. Tabla “alertas”:

La tabla alertas es una de las tablas más importantes para el sistema ya que en esta se almacenan las distintas, alertas que han sido reportadas en la institución educativa. La tabla alertas es utilizada es mencionada en las funciones: cantidad de emergencias, consulta de registros del historial, registro de alertas y actualizar estado de alerta. La cual se muestra a continuación en la siguiente captura:



|   | id_alerta<br>[PK] integer | id_usuario<br>integer | id_tipo<br>integer | id_ubicacion<br>integer | comando_recibido<br>character varying (50) | descripcion<br>character varying (300)                       | estado<br>character varying (50) | hora_emerg<br>time without t |
|---|---------------------------|-----------------------|--------------------|-------------------------|--------------------------------------------|--------------------------------------------------------------|----------------------------------|------------------------------|
| 1 | 4                         | 4                     | 1                  | 1                       | Incendio,Edif_A,Activa                     | Alarma de incendio activada en pasillo del Edificio A        | ATENDIDA                         | 12:20:00                     |
| 2 | 5                         | 3                     | 2                  | 1                       | Medica,Edif_A,Activa                       | Desmayo de alumno en Primer Año Sección "A" del Edific...    | FALSA                            | 15:10:00                     |
| 3 | 7                         | 3                     | 2                  | 2                       | Medica,Edif_B,Activa                       | Estudiante lesionado en la cancha.                           | FALSA                            | 05:10:00                     |
| 4 | 2                         | 2                     | 2                  | 2                       | Medica,Edif_B,Activa                       | Estudiante herido en la cancha central del Edificio B        | ATENDIDA                         | 04:30:00                     |
| 5 | 3                         | 1                     | 3                  | 3                       | Seguridad,Edif_C,En_proce...               | Persona no autorizada detectada en el acceso al Edifici...   | EN_PROCESO                       | 11:45:00                     |
| 6 | 1                         | 1                     | 1                  | 1                       | Incendio,Edif_A,Atendida                   | Humo detectado cerca del laboratorio principal en Edifici... | ATENDIDA                         | 10:30:00                     |
| 7 | 6                         | 1                     | 1                  | 2                       | Incendio,Edif_B,Atendida                   | Fuga de gas detectada en el área de cocina del Edificio B    | ATENDIDA                         | 15:10:00                     |

Figura 3 Captura de evidencia de tabla alertas en PostgreSQL



### 9.1.2. Tabla “instituciones”:

La tabla instituciones es utilizada para almacenar la información de las instituciones educativas que forman parte del sistema. Esta tabla es mencionada en las funciones de consulta de datos de usuarios, consultar institución educativa y registrar institución educativa, La cual se muestra a continuación en la siguiente captura:

The screenshot shows a PostgreSQL query interface. The query executed is `SELECT * FROM public.instituciones;`. The results are displayed in a table with the following structure:

|   | id_institucion<br>[PK] integer | nombre<br>character varying (125) | creada<br>date |
|---|--------------------------------|-----------------------------------|----------------|
| 1 | 1                              | Instituto Católico Karol Wojtyła  | 2026-09-09     |

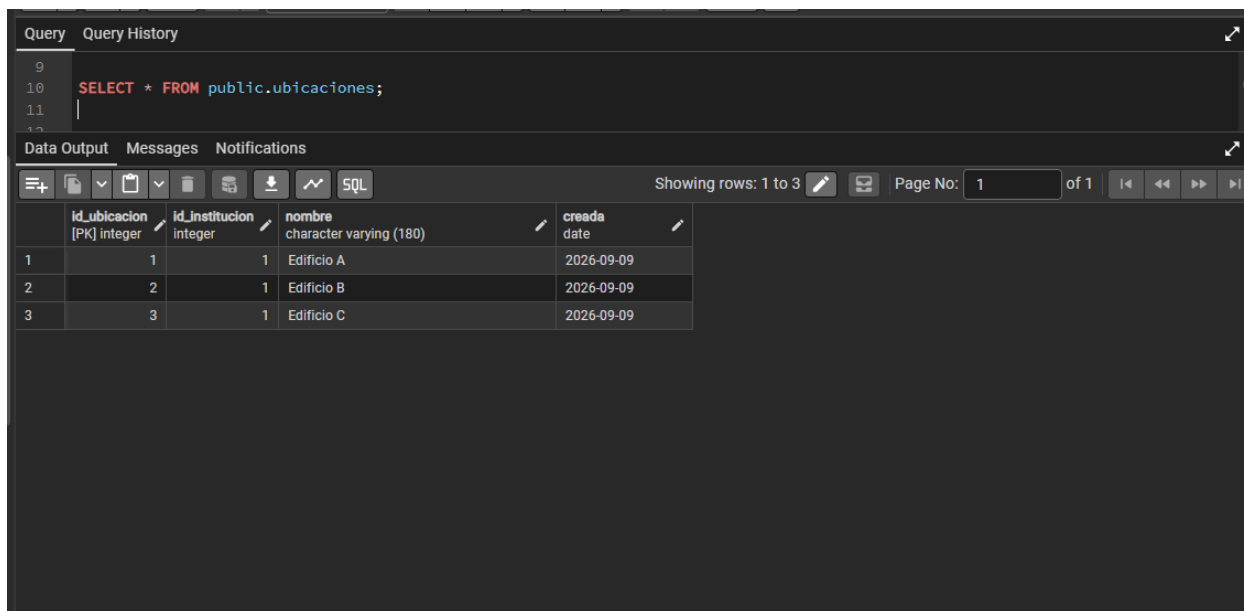
Below the table, a green status bar indicates: "Successfully run. Total query runtime: 175 msec. 1 rows affected."

Figura 4 Captura de evidencia de tabla instituciones en PostgreSQL

### 9.1.3. Tabla “ubicaciones”:

La tabla ubicaciones es utilizada para almacenar las diferentes ubicaciones o edificios que se encuentran registrados dentro de la institución educativa. Esta tabla es mencionada en las funciones de cantidad de ubicaciones, consulta de registros del historial, consulta de alertas, consultar ubicaciones y registrar una ubicación.





Query Query History

```
9  
10 SELECT * FROM public.ubicaciones;  
11  
12
```

Data Output Messages Notifications

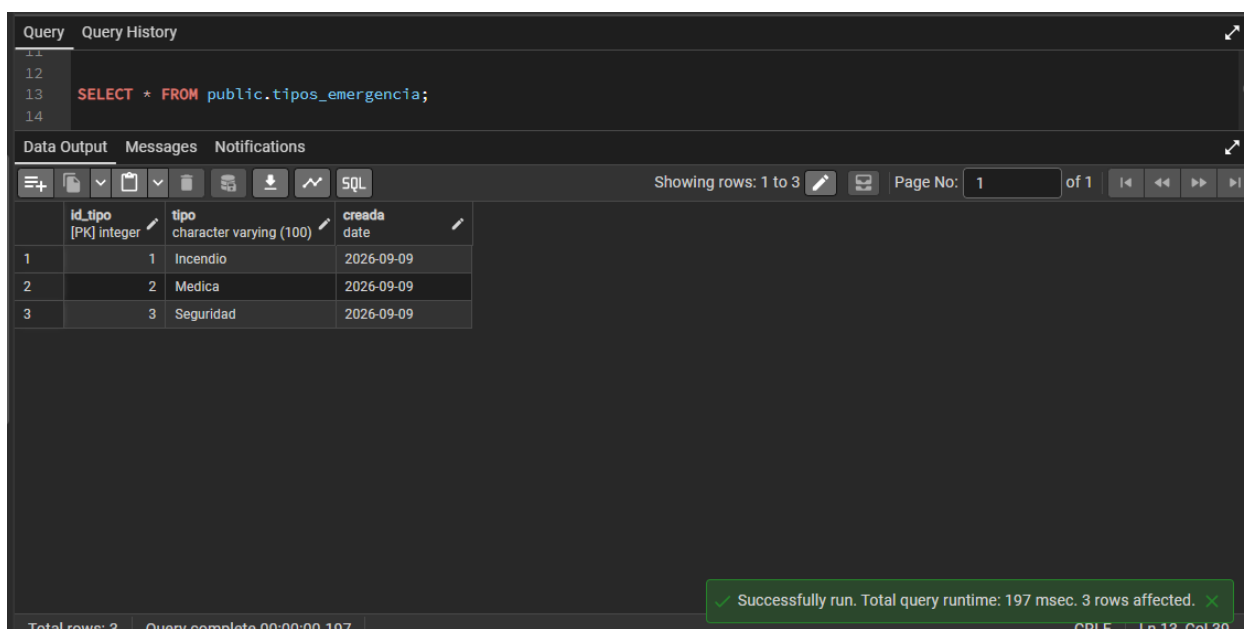
Showing rows: 1 to 3 Page No: 1 of 1

|   | id_ubicacion<br>[PK] integer | id_institucion<br>integer | nombre<br>character varying (180) | creada<br>date |
|---|------------------------------|---------------------------|-----------------------------------|----------------|
| 1 | 1                            | 1                         | Edificio A                        | 2026-09-09     |
| 2 | 2                            | 1                         | Edificio B                        | 2026-09-09     |
| 3 | 3                            | 1                         | Edificio C                        | 2026-09-09     |

Figura 5 Captura de evidencia de tabla ubicaciones en PostgreSQL

#### 9.1.4. Tabla “tipos\_emergencia”:

La tabla tipos\_emergencia es utilizada para almacenar los diferentes tipos de emergencias que el sistema puede gestionar, como médicas, de seguridad y de incendio. Esta tabla es mencionada en las funciones de consulta de registros del historial, consulta de alertas, consultar tipos de emergencia y registrar un tipo de emergencia.



Query Query History

```
11  
12  
13 SELECT * FROM public.tipos_emergencia;  
14
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

|   | id_tipo<br>[PK] integer | tipo<br>character varying (100) | creada<br>date |
|---|-------------------------|---------------------------------|----------------|
| 1 | 1                       | Incendio                        | 2026-09-09     |
| 2 | 2                       | Medica                          | 2026-09-09     |
| 3 | 3                       | Seguridad                       | 2026-09-09     |

✓ Successfully run. Total query runtime: 197 msec. 3 rows affected. ✕

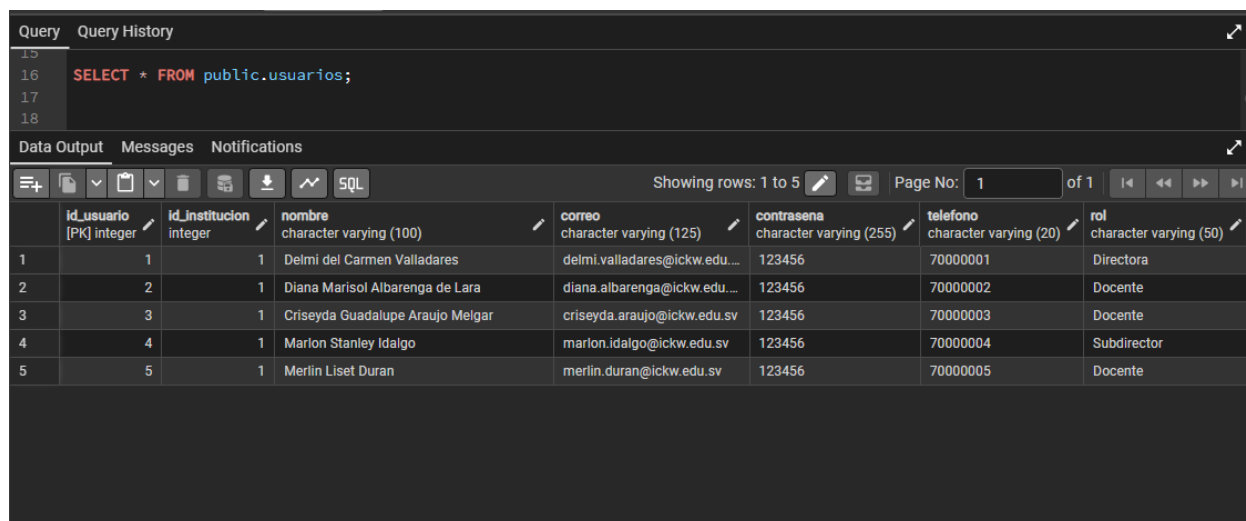
Total rows: 3 Query complete 00:00:00.197

Figura 6 Captura de evidencia de tabla tipos\_emergencia en PostgreSQL



### 9.1.5. Tabla “usuarios”:

La tabla usuarios es una de las tablas importantes para el funcionamiento del sistema, ya que en esta se almacena los datos de los usuarios. Esta tabla es mencionada en las funciones de autenticación de usuarios, registro de usuarios nuevos, consulta de datos de usuarios, cantidad de docentes registrados, cantidad de usuarios registrados, consulta de registros del historial y consulta de alertas.



|   | id_usuario<br>[PK] integer | id_institucion<br>integer | nombre<br>character varying (100) | correo<br>character varying (125) | contrasena<br>character varying (255) | telefono<br>character varying (20) | rol<br>character varying (50) |
|---|----------------------------|---------------------------|-----------------------------------|-----------------------------------|---------------------------------------|------------------------------------|-------------------------------|
| 1 | 1                          | 1                         | Delmi del Carmen Valladares       | delmi.valladares@ickw.edu...      | 123456                                | 70000001                           | Directora                     |
| 2 | 2                          | 1                         | Diana Marisol Albarenga de Lara   | diana.albarenga@ickw.edu...       | 123456                                | 70000002                           | Docente                       |
| 3 | 3                          | 1                         | Criseyda Guadalupe Araujo Melgar  | criseyda.araujo@ickw.edu.sv       | 123456                                | 70000003                           | Docente                       |
| 4 | 4                          | 1                         | Marlon Stanley Idalgo             | marlon.idalgo@ickw.edu.sv         | 123456                                | 70000004                           | Subdirector                   |
| 5 | 5                          | 1                         | Merlin Liset Duran                | merlin.duran@ickw.edu.sv          | 123456                                | 70000005                           | Docente                       |

Figura 7 Captura de evidencia de tabla usuarios en PostgreSQL

### 9.1.6. Tabla “historial”:

La tabla historial es utilizada para almacenar los diferentes registros y eventos relacionados con las emergencias que han ocurrido en el sistema. Esta tabla es mencionada en las funciones de cantidad de registros de historial y consulta de registros del historial.

Query

Query History

```

18
19 SELECT * FROM public.historial;
20

```

Data Output

Messages

Notifications

+

📄

▼

📄

▼

🗑️

📄

📄

📄

📄

SQL

Showing rows: 1 to 7

Page No: 1 of 1

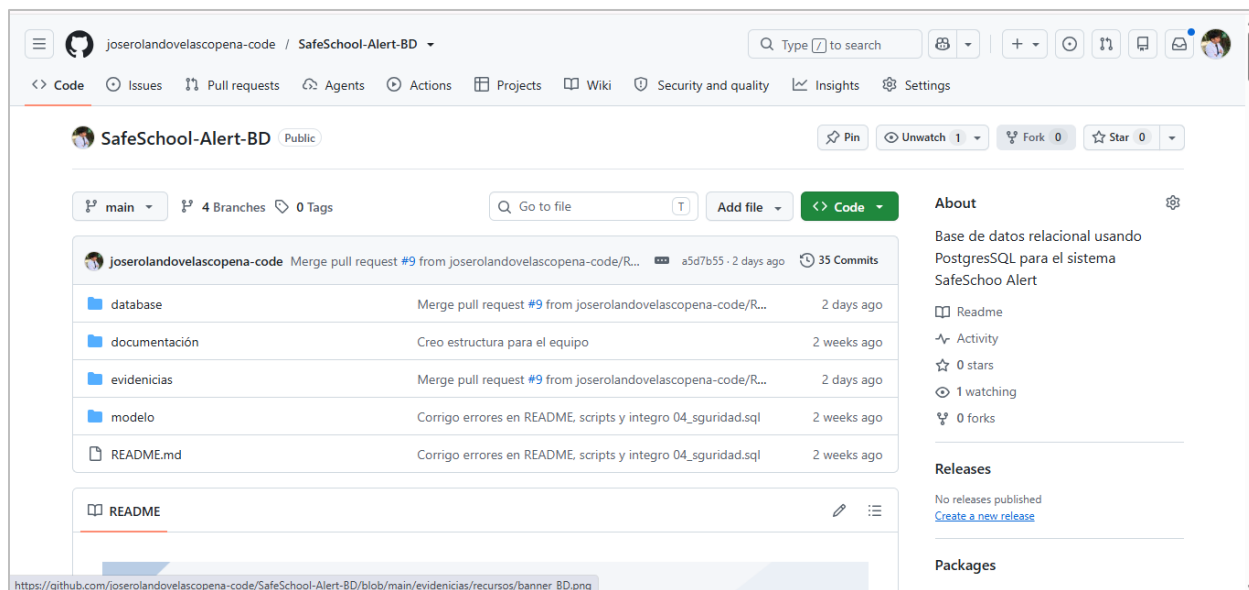
|   | id_historial<br>[PK] integer | id_usuario<br>integer | id_alerta<br>integer | id_tipo<br>integer | id_ubicacion<br>integer | descripcion<br>character varying (200)                       | fecha_hora<br>timestamp without time zone |
|---|------------------------------|-----------------------|----------------------|--------------------|-------------------------|--------------------------------------------------------------|-------------------------------------------|
| 1 | 1                            | 1                     | 1                    | 1                  | 1                       | Humo detectado cerca del laboratorio principal en Edifici... | 2026-09-26 16:51:34.717583                |
| 2 | 2                            | 2                     | 2                    | 2                  | 2                       | Estudiante herido en la cancha central del Edificio B.       | 2026-09-26 16:51:34.717583                |
| 3 | 3                            | 3                     | 3                    | 3                  | 3                       | Persona no autorizada detectada en el acceso al Edificio ... | 2026-09-26 16:51:34.717583                |
| 4 | 4                            | 4                     | 4                    | 1                  | 1                       | Alarma de incendio activada en pasillo del Edificio A.       | 2026-09-26 16:51:34.717583                |
| 5 | 5                            | 3                     | 5                    | 2                  | 1                       | Desmayo de alumno en el primero Sección "A" del Edifici...   | 2026-09-26 16:51:34.717583                |
| 6 | 6                            | 1                     | 6                    | 1                  | 2                       | Humo detectado cerca del laboratorio principal en Edifici... | 2026-09-26 16:51:34.717583                |
| 7 | 7                            | 3                     | 7                    | 2                  | 2                       | Estudiante lesionado en la cancha.                           | 2026-09-26 16:51:34.717583                |

*Figura 8 Captura de evidencia de tabla historial en PostgreSQL*

## 9.2. GitHub

### 9.2.1. Repositorio del proyecto:

A continuación, se presenta el repositorio del proyecto, en el cual se almacenan los archivos trabajados durante las diferentes semanas de trabajo. Este nos permite llevar un historial de las versiones del proyecto y regresar a versiones anteriores en caso de que algo salga mal o se genere algún error. De igual manera, en este se almacena la documentación realizada del proyecto, el modelo físico y posteriormente la documentación de correspondencia de la BD.



*Figura 9 Captura del repositorio del proyecto en GitHub*

### 9.2.2. Ramas de los integrantes.

En este apartado se presenta la evidencia de las ramas de trabajo creadas en el repositorio del proyecto, en las cuales cada integrante que conforma el proyecto SafeSchool Alert ha trabajado y desarrollado las diferentes actividades prácticas, de preparación y correcciones de la estructura y datos de la base de datos. Además, estas ramas nos permiten trabajar de manera colaborativa las diferentes partes del proyecto, según el rol de cada integrante.

| N° | Integrante                      | Rama                  |
|----|---------------------------------|-----------------------|
| 1  | José Rolando Velasco Peña       | Rolando_SafeSchool_DB |
| 2  | Luis Mario Meléndez Escobar     | Luis_SafeSchool_DB    |
| 3  | Ulises de Jesús Mercado Alberto | ulises_SafeSchool_DB  |

| Default               |            |              |        |       |              |
|-----------------------|------------|--------------|--------|-------|--------------|
| Branch                | Updated    | Check status | Behind | Ahead | Pull request |
| main                  | 2 days ago |              |        |       | Default      |
| Your branches         |            |              |        |       |              |
| Branch                | Updated    | Check status | Behind | Ahead | Pull request |
| Rolando_SafeSchool_DB | 2 days ago |              | 14     | 0     | #9           |
| Active branches       |            |              |        |       |              |
| Branch                | Updated    | Check status | Behind | Ahead | Pull request |
| Rolando_SafeSchool_DB | 2 days ago |              | 14     | 0     | #9           |
| Luis_SafeSchool_DB    | 2 days ago |              | 10     | 5     |              |
| ulises_SafeSchool_DB  | last week  |              | 21     | 0     | #8           |

© 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

*Figura 10 Captura de las ramas de cada uno de los integrantes del equipo*

### 9.2.3. Commits realizados.

Para evidenciar el trabajo realizado por los diferentes integrantes, a continuación, se presenta mediante una captura de pantalla la lista de commits, en la cual se muestran los diferentes aportes y correcciones que cada integrante ha realizado al proyecto. Estos commits representan una versión del proyecto y además nos permiten realizar aportaciones y trabajar de manera colaborativa en el mismo proyecto:

|                                                                                                                     |  |         |
|---------------------------------------------------------------------------------------------------------------------|--|---------|
| Integro al proyecto 4 consultas utilizado INNER JOIN en alertas<br>joserolandovelascopena-code committed 5 days ago |  | dacffc7 |
| Commits on Sep 18, 2026                                                                                             |  |         |
| Subo mi documento de evidencias de la semana al repositorio<br>joserolandovelascopena-code committed last week      |  | 978b98e |
| Commits on Sep 17, 2026                                                                                             |  |         |
| Merge pull request #8 from joserolandovelascopena-code/ulises_SafeSchool_DB<br>3385527-ulises authored last week    |  | 6c623c8 |
| Agrego archivo consultas usuarios<br>3385527-ulises committed last week                                             |  | 06beb9b |
| Commits on Sep 14, 2026                                                                                             |  |         |
| Merge pull request #7 from joserolandovelascopena-code/Luis_SafeSchool_DB<br>lets89 authored 2 weeks ago            |  | ffc458e |
| Merge pull request #6 from joserolandovelascopena-code/main<br>lets89 authored 2 weeks ago                          |  | 0e47e64 |
| Agrego archivo datos prueba usuarios<br>3385527-ulises committed 2 weeks ago                                        |  | a444563 |

*Figura 11 Captura de lista de commits de los integrantes*



#### **9.2.4. Archivos incorporados a la carpeta integración.**

## **10. CONCLUSIÓN**